

DAY 1

What is Apache Spark

Apache Spark is an open-source, distributed computing framework designed for fast processing of large-scale data. It allows data to be processed in parallel across a cluster of machines, making it much faster than traditional systems like Hadoop MapReduce. Spark supports multiple workloads such as batch processing, real-time streaming, machine learning, and graph processing, all within a single platform. Its in-memory computation capability significantly improves performance, especially for iterative tasks.

Eg: Imagine you have **1 TB of log data**. A single machine would take hours. Spark splits it.

Why we use PySpark

PySpark is the Python API for Apache Spark, allowing developers to use Python to work with big data instead of Scala or Java. It is widely used because Python is easy to learn and has a rich ecosystem of libraries. PySpark enables users to write distributed data processing code in a simple and readable way while still leveraging Spark's powerful engine. It is especially popular among data analysts and data scientists who want to combine big data processing with tools like Pandas and machine learning libraries.

PySpark lets you use Python instead of Scala.

Difference between SQL and PySpark

SQL is a declarative language used to query and manipulate structured data, mainly in relational databases. Users specify *what* they want, and the system decides *how* to execute it. In contrast, PySpark is a programming interface where you explicitly define transformations and actions on data using Python. SQL is simpler and ideal for straightforward queries, aggregations, and reporting, while PySpark is more flexible and powerful for complex data transformations, large-scale processing, and integration with advanced analytics. Spark also provides Spark SQL, which bridges both worlds by allowing SQL queries on distributed datasets.

- SQL = simple, readable, limited flexibility
 - PySpark = more control, scalable, complex pipelines
-

What is a DataFrame?

A DataFrame in Apache Spark is a distributed collection of data organized into rows and

columns, like a table in a database or a spreadsheet. It provides a high-level abstraction for working with structured and semi-structured data. DataFrames support operations like filtering, grouping, and aggregation, and they are optimized by Spark's internal engine for efficient execution. They are widely used because they combine ease of use with strong performance and scalability.

What is Lazy Evaluation?

Lazy evaluation in Apache Spark means that transformations on data (like filtering or mapping) are not executed immediately when they are defined. Instead, Spark builds a logical plan of all operations and only executes them when an action (such as collecting results or saving data) is called. This approach allows Spark to optimize the entire workflow, reducing unnecessary computations and improving performance. Lazy evaluation is a key reason why Spark can handle large datasets efficiently.

Spark Session:

A **Spark Session** in Apache Spark is the **entry point** to work with Spark. It acts as the central object that lets you connect to a Spark cluster and perform all operations like reading data, transforming it, and running queries.

Driver vs Workers

- **Driver** → controls execution (brain)
- **Workers** → process data (do actual work)

In Apache Spark, the architecture is divided between the Driver and Worker nodes, each with a distinct role. The Driver acts as the “brain” of the application—it is responsible for creating the Spark session, defining transformations and actions, building the execution plan (DAG), and coordinating the overall workflow. On the other hand, Workers are the machines that perform the computation. They execute the tasks assigned by the Driver, process the data in parallel, and return the results. In simple terms, the Driver decides *what* needs to be done, while the Workers handle *how* the data is processed at scale.

Action vs Transformation

- **Transformations** → `select()`, `filter()`, `withColumn()` (lazy)
Transformations such as `select()`, `filter()`, and `withColumn()` are *lazy*, meaning they do not immediately process data; instead, they define a logical

plan or sequence of operations to be performed later. Spark simply records these steps in a DAG (Directed Acyclic Graph) without executing them right away.

- **Actions** → `show()`, `count()`, `collect()` (trigger execution)
Actions like `show()`, `count()`, and `collect()` are what trigger *execution*. When an action is called, Spark evaluates all the previously defined transformations, optimizes the execution plan, and then runs the computation to produce a result. This lazy evaluation model helps Spark optimize performance by minimizing unnecessary data processing and combining operations efficiently.

```
df_patient = spark.table("workspace.new.data_2")
```

```
df_patient.show()
```

DAY 2:

`Limit()`

`head()`

`describe()`

`Filter()` → `df_stud.filter((df_stud.age > 20) & (df_stud.course == "BCA")).display()`

Relational operations: &, |, ~

Logical operations: >, <, >=, <=, ==, !=

Aggregate Functions: `df_stud.agg({"marks": "sum"}).show()`

`count()` → counts non-null rows

`countDistinct()` → counts distinct values

`sum()` → total of a column

`avg()` or `mean()` → average value

`min()` → smallest value

`max()` → largest value

`stddev()` → Standard deviation

`variance()` → Variance

`first()` → first value in a group

`last()` → last value in a group

`collect_list()` → collects values into a list

`collect_set()` → collects unique values

`approx_count_distinct()` → fast approximate distinct count

`sumDistinct()` → sum of distinct values

skewness() → data skewness
kurtosis() → data kurtosis

Business Use of Aggregations

Aggregations in PySpark are widely used in business scenarios to summarize large volumes of data into meaningful insights that support decision-making. For example, companies use functions like `sum()`, `avg()`, `count()`, `min()`, and `max()` along with `groupBy()` to analyze sales performance, such as calculating total revenue by region, average customer spending, or the number of transactions per day. In retail, aggregations help identify top-selling products and seasonal trends, while in finance they are used to compute metrics like total expenses, profit margins, and risk exposure. Marketing teams rely on aggregations to measure campaign effectiveness, such as conversion rates and customer segmentation. By condensing raw data into key performance indicators (KPIs), aggregations enable businesses to make faster, data-driven decisions and uncover patterns that would be difficult to detect in unprocessed datasets.

DAY 3

Difference between `display()` and `show()`? use cases of both

The **`show()`** method is a built-in PySpark DataFrame function that prints rows in a simple text table directly to the console, making it reliable in any environment such as scripts or terminals; it's typically used for quick debugging, checking schema outputs, or verifying small samples during data processing. In contrast, **`display()`** is not part of core PySpark but is provided by notebook environments like Databricks, where it renders DataFrames in an interactive, visually rich format with features like sorting, filtering, and charting; this makes it more suitable for exploratory data analysis, presentations, and when you need to visually inspect patterns or trends in the data.

Developers use `show()` for lightweight, code-centric tasks and `display()` when working interactively and needing deeper insights through UI-based exploration.

```
data = [
    ("P001", "Arjun", 45, "Male", "Cardiology", "Kochi"),
    ("P002", "Meera", 32, "Female", "Neurology", "Kozhikode")
]
columns = ["patient_id", "name", "age", "gender", "department", "city"]
df = spark.createDataFrame(data, columns)
df.display()
```

> See performance (1)

> df: pyspark.sql.connect.dataframe.DataFrame = [patient_id: string, name: string ... 4 more fields]

	patient_id	name	age	gender	department	city
1	P001	Arjun	45	Male	Cardiology	Kochi
2	P002	Meera	32	Female	Neurology	Kozhikode

```
data = [
    ("P001", "Arjun", 45, "Male", "Cardiology", "Kochi"),
    ("P002", "Meera", 32, "Female", "Neurology", "Kozhikode")
]
columns = ["patient_id", "name", "age", "gender", "department", "city"]
df = spark.createDataFrame(data, columns)
df.show()
```

> See performance (1)

> df: pyspark.sql.connect.dataframe.DataFrame = [patient_id: string, name: string ... 4 more fields]

patient_id	name	age	gender	department	city
P001	Arjun	45	Male	Cardiology	Kochi
P002	Meera	32	Female	Neurology	Kozhikode

Functions

withColumn()
when() , otherwise()
lit()

Example:

```
from pyspark.sql.functions import when,lit
df_patient.withColumn("result",when(df_patient.age<30,"Young").when((df_patient.age
>=30) & (df_patient.age <50),"Middle").otherwise("senior")).show()
df_patient.withColumn("age_plus_10", df_patient.age + 10).show()
df_patient.withColumn("country",lit("India")).show()
```

SQL → PySpark Mapping Table

SQL	PySpark
SELECT	select()
WHERE	filter()
AND	&
OR	
Not	~
If...else	When() otherwise()
ALIAS	alias()
LIMIT	limit()
ORDER BY	orderBy() / sort()
HAVING	filter() after groupBy()
WITH COLUMN	withColumn()

DAY 4

Functions:

groupBy() -> group the columns
orderBy() -> ascending or descending order

What I Learned This Week?

This week, I built a strong foundation in PySpark and understood how it connects closely with SQL concepts while enabling large-scale data processing. I learned that PySpark can be viewed as SQL in Python form, where a `DataFrame` behaves like a table with rows and columns. I explored how common SQL operations map to PySpark functions, such as `select()` for choosing columns and `filter()` for applying conditions, along with logical operators like `&` and `|`. A key concept I understood is that transformations are lazy—they only define a plan and do not execute immediately—while actions like `show()` or `count()` trigger actual computation. I also learned that real data processing work happens when transformations are combined with aggregations like `groupBy()` and functions such as `sum()` or `avg()` to extract meaningful insights. Additionally, I gained clarity on Spark's architecture, where the Driver acts as the brain that controls execution and builds the plan, while Worker nodes perform the actual data processing in parallel. Overall, this week helped me understand both the theoretical concepts and practical workflow of working with big data in PySpark.
